

Programação Orientada a Objetos II

Reflexão e introspecção; Aplicação de reflexão

Conceito de reflexão

- **Reflexão:** Capacidade de um programa examinar e modificar sua própria estrutura e comportamento em tempo de execução
- **API de Reflexão:** Conjunto de classes e métodos que permitem introspecção de classes, métodos, campos e construtores
- **Metadados:** Informações sobre a estrutura do código (classes, métodos, atributos) disponíveis em runtime
- **Class object:** Objeto que representa uma classe em tempo de execução, permitindo acesso aos seus metadados

Exemplo prático – Conceito de reflexão

```
public class ExemploReflexao {  
    public static void main(String[] args) {  
        // Obtendo informações da classe String  
        Class<?> clazz = String.class;  
  
        System.out.println("Classe: " + clazz.getName());  
        System.out.println("Métodos: " + clazz.getMethods().length);  
        System.out.println("Campos: " + clazz.getFields().length);  
  
        // Verificando se é uma classe ou interface  
        System.out.println("É interface? " + clazz.isInterface());  
    }  
}
```

Introspecção em Java

- **Introspecção:** Capacidade de examinar a estrutura interna de objetos e classes em tempo de execução
- **Class.forName():** Método para obter o objeto Class de uma classe pelo seu nome
- **getClass():** Método que retorna o objeto Class de uma instância
- **getMethods():** Obtém array com todos os métodos públicos da classe
- **getFields():** Obtém array com todos os campos públicos da classe
- **getConstructors():** Obtém array com todos os construtores públicos da classe

Exemplo prático – Introspecção em Java

```
class Pessoa {  
    private String nome;  
    public int idade;  
  
    public void falar() { System.out.println("Falando"); }  
}
```

```
public class ExemploIntrospecao {  
    public static void main(String[] args) throws Exception {  
        Pessoa p = new Pessoa();  
        Class<?> clazz = p.getClass();  
  
        // Listar campos  
        System.out.println("Campos:");  
        for (Field f : clazz.getDeclaredFields()) {  
            System.out.println("- " + f.getName());  
        }  
  
        // Listar métodos  
        System.out.println("Métodos:");  
        for (Method m : clazz.getDeclaredMethods()) {  
            System.out.println("- " + m.getName());  
        }  
    }  
}
```

Manipulação de classes em runtime

Conteúdo teórico detalhado sobre **Manipulação de classes em runtime**.

Definições

- **Invocação dinâmica:** Chamada de métodos através de reflexão usando `Method.invoke()`
- **Instanciação dinâmica:** Criação de objetos usando `Constructor.newInstance()`
- **Acesso a campos:** Leitura e escrita de atributos usando `Field.get()` e `Field.set()`
- **Modificadores de acesso:** Controle de visibilidade (`public`, `private`, `protected`) que pode ser contornado com `setAccessible()`
- **ClassLoader:** Mecanismo responsável por carregar classes dinamicamente

Exemplo prático – Manipulação de classes em runtime

```
class Calculadora {  
    private int resultado = 0;  
  
    public int somar(int a, int b) {  
        return resultado = a + b;  
    }  
}
```

```
public class ExemploManipulacao {  
    public static void main(String[] args) throws Exception {  
        // Criação dinâmica  
        Class<?> clazz = Calculadora.class;  
        Object calc = clazz.getDeclaredConstructor().newInstance();  
  
        // Invocação dinâmica de método  
        Method somar = clazz.getMethod("somar", int.class, int.class);  
        Object resultado = somar.invoke(calc, 10, 5);  
        System.out.println("Resultado: " + resultado);  
  
        // Acesso a campo privado  
        Field campo = clazz.getDeclaredField("resultado");  
        campo.setAccessible(true);  
        System.out.println("Campo privado: " + campo.get(calc));  
    }  
}
```

Casos de uso

- **Frameworks:** Utilizam reflexão para injeção de dependência e mapeamento objeto-relacional
- **Serialização:** Conversão de objetos para formatos como JSON, XML usando reflexão
- **Testes unitários:** Ferramentas como JUnit usam reflexão para descobrir e executar métodos de teste
- **Anotações:** Processamento de metadados em tempo de execução (@Entity, @Service, etc.)
- **Plugins:** Carregamento dinâmico de funcionalidades sem conhecimento prévio das classes
- **APIs genéricas:** Criação de código que funciona com qualquer tipo de objeto

Exemplo prático – Casos de uso

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@interface Executar { }

class Servico {
    @Executar
    public void tarefa1() {
        System.out.println("Executando tarefa 1");
    }

    public void tarefa2() {
        System.out.println("Executando tarefa 2");
    }
}
```

```
public class ExemploCasoUso {
    public static void main(String[] args) throws Exception {
        Servico servico = new Servico();

        // Executar apenas métodos anotados
        for (Method m : servico.getClass().getMethods()) {
            if (m.isAnnotationPresent(Executar.class)) {
                m.invoke(servico);
            }
        }
    }
}
```

Boas práticas

Conteúdo teórico detalhado sobre **Boas práticas**.

- **Cache de objetos Class:** Evitar chamadas repetidas a `Class.forName()` armazenando resultados
- **Tratamento de exceções:** Capturar adequadamente `ReflectiveOperationException` e suas subclasses
- **Validação de segurança:** Verificar permissões antes de usar `setAccessible()`
- **Performance:** Usar reflexão apenas quando necessário, pois é mais lenta que chamadas diretas
- **Tipo de retorno:** Fazer cast seguro dos resultados de `invoke()` e `newInstance()`
- **Documentação:** Documentar claramente o uso de reflexão no código para facilitar manutenção

Exemplo prático – Boas práticas

```
import java.util.*;

public class ExemploBoasPraticas {
    // Cache para evitar Class.forName repetido
    private static final Map<String, Class<?>> cache = new HashMap<>();

    public static void main(String[] args) {
        try {
            // Boa prática: Cache de classes
            Class<?> stringClass = getClassSafely("java.lang.String");

            // Boa prática: Tratamento de exceções
            Object resultado = invokeMethodSafely("test", "length");
            System.out.println("Comprimento: " + resultado);
        } catch (Exception e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }
}
```

```
private static Class<?> getClassSafely(String className) {
    return cache.computeIfAbsent(className, name -> {
        try {
            return Class.forName(name);
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("Classe não encontrada: " + name);
        }
    });
}

private static Object invokeMethodSafely(Object target, String methodName) {
    try {
        return target.getClass().getMethod(methodName).invoke(target);
    } catch (Exception e) {
        throw new RuntimeException("Erro ao invocar: " + methodName, e);
    }
}
```

Exercício – Reflexão e introspecção

1. No exercício sobre o padrão de Projeto Strategy:
 - a) Criar uma anotação personalizada chamada `@Implementacao` que possa ser usada para marcar classes que implementam a interface da estratégia.
 - b) Altere o código para usar reflexão na escolha da classe concreta.
 - c) O código deve buscar todas as classes anotadas com `@Implementacao` e criar uma instância de qualquer uma delas com base em uma string que representa o nome da classe.
 - d) Para buscar todas as classes anotadas, você pode usar a biblioteca `Reflections`.

Referências

- Page-Jones, M. *Fundamentos do Desenho Orientado a Objeto com UML*. Makron Books.
- Larman, C. *Utilizando UML e Padrões*. Saraiva.
- Ementa da disciplina.